

Exercise 7 – Debug ABAP Code

Unit 2 / Step-by-step Practice Documentation

Status: COMPLETED ✓

1. What is this exercise about?

This exercise teaches how to use the ABAP debugger in Eclipse ADT. Instead of only looking at the final console output, the debugger lets us stop the program while it is running, inspect variable values, execute statements one at a time, and observe how the values change.

2. Problem → Solution

Problem:

The program calculates a loan repayment plan, but simply seeing the final result does not tell us whether every calculation happened correctly. We need a way to inspect the program while it runs.

Solution:

Use Eclipse ADT's ABAP Debugger. A breakpoint stops execution at a selected line. The Variables/Locals view shows the current values. F5/F6 can execute the program step by step, and watchpoints can stop execution when a variable changes.

3. Class Used

Custom class: ZCL_9309_DEBUG

Template: /LRN/CL_S4D400_BTT_DEBUG

4. Important ABAP Concepts

Term	Simple meaning
Breakpoint	A place where the debugger pauses program execution.
Watchpoint	Stops the debugger when the value of a selected variable changes.
Variables / Locals	Shows the current runtime values of variables.
F5 – Step Into	Executes the current statement and moves to the next execution point.
F6 – Step Over	Executes the current statement/code block without entering reusable code in detail.
F8 – Resume	Continues execution until another

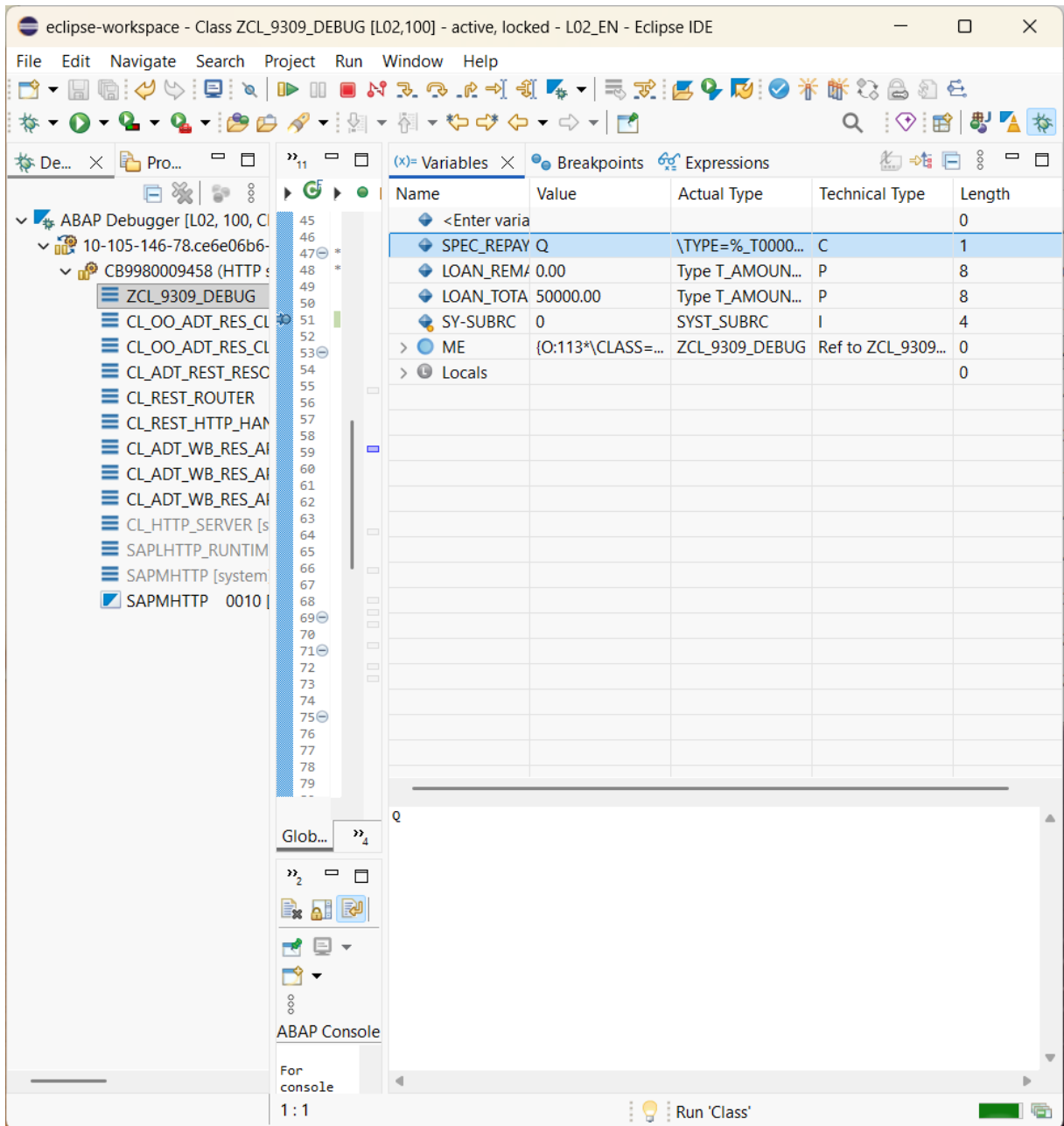
	breakpoint/watchpoint is reached.
APPEND	Adds a new row to an internal table.
CASE	Chooses one branch based on a value.

5. Task 1 – Preparation

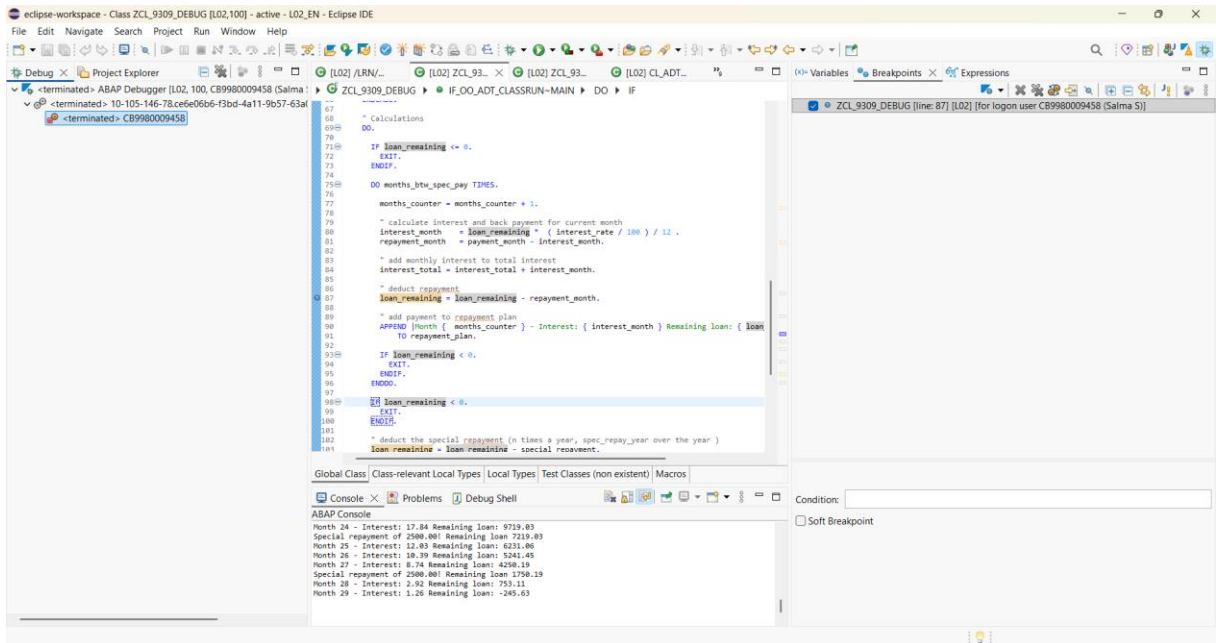
The template class was copied into the personal package and renamed ZCL_9309_DEBUG. The class implements IF_OO_ADT_CLASSRUN so it can be executed as an Eclipse console application.

Solution steps:

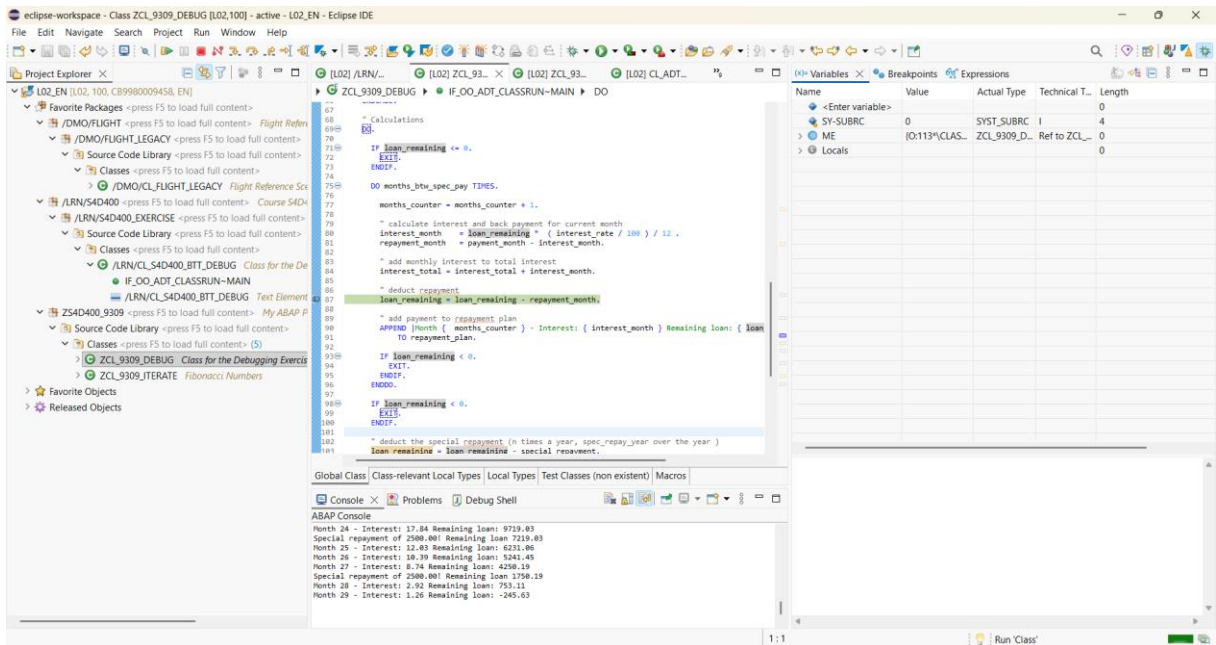
1. Open /LRN/CL_S4D400_BTT_DEBUG.
2. Link the Project Explorer with the editor.
3. Duplicate the class into package ZS4D400_9309.
4. Use the class name ZCL_9309_DEBUG.
5. Select the existing ABAP Exercises transport request.
6. Activate the class with Ctrl + F3.
7. Run it with F9 and verify the console output.



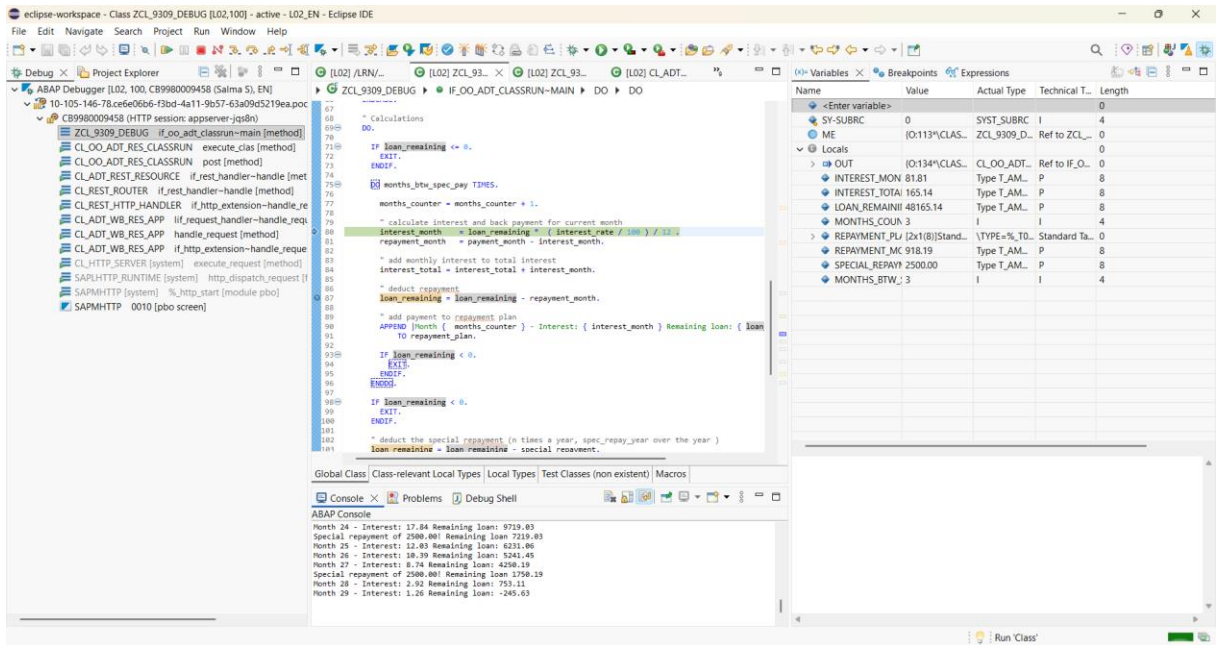
Screenshot: ZCL_9309_DEBUG class open and ready for debugging.



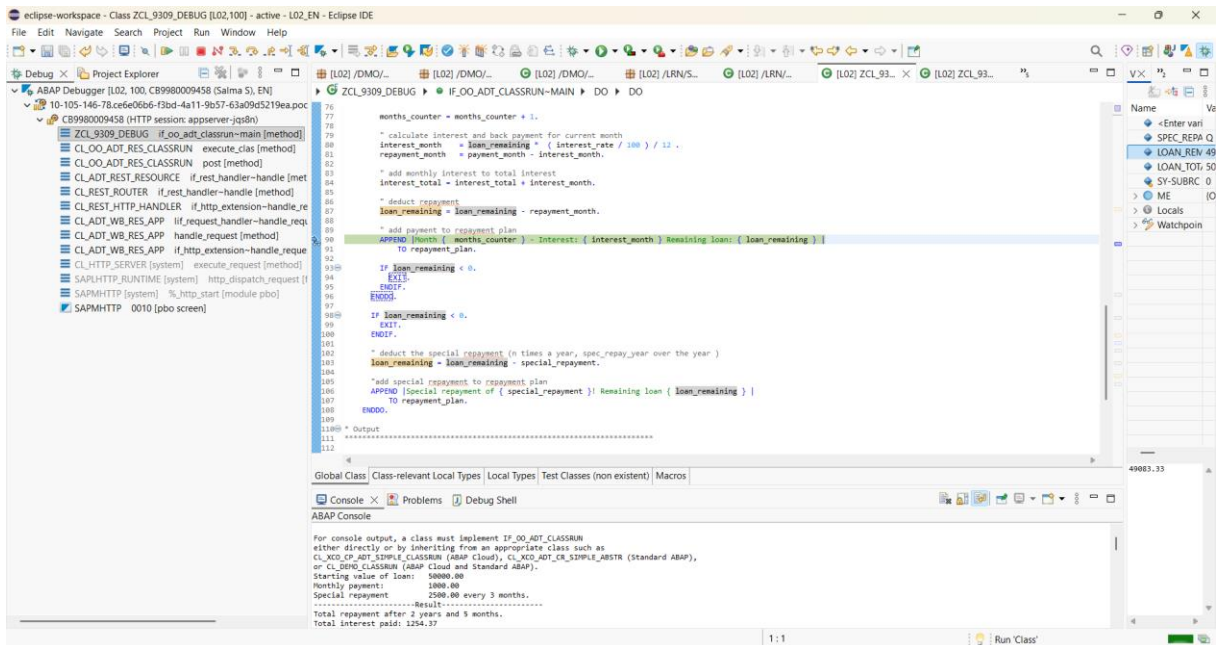
Screenshot: Breakpoint reached in ZCL_9309_DEBUG at the repayment calculation.



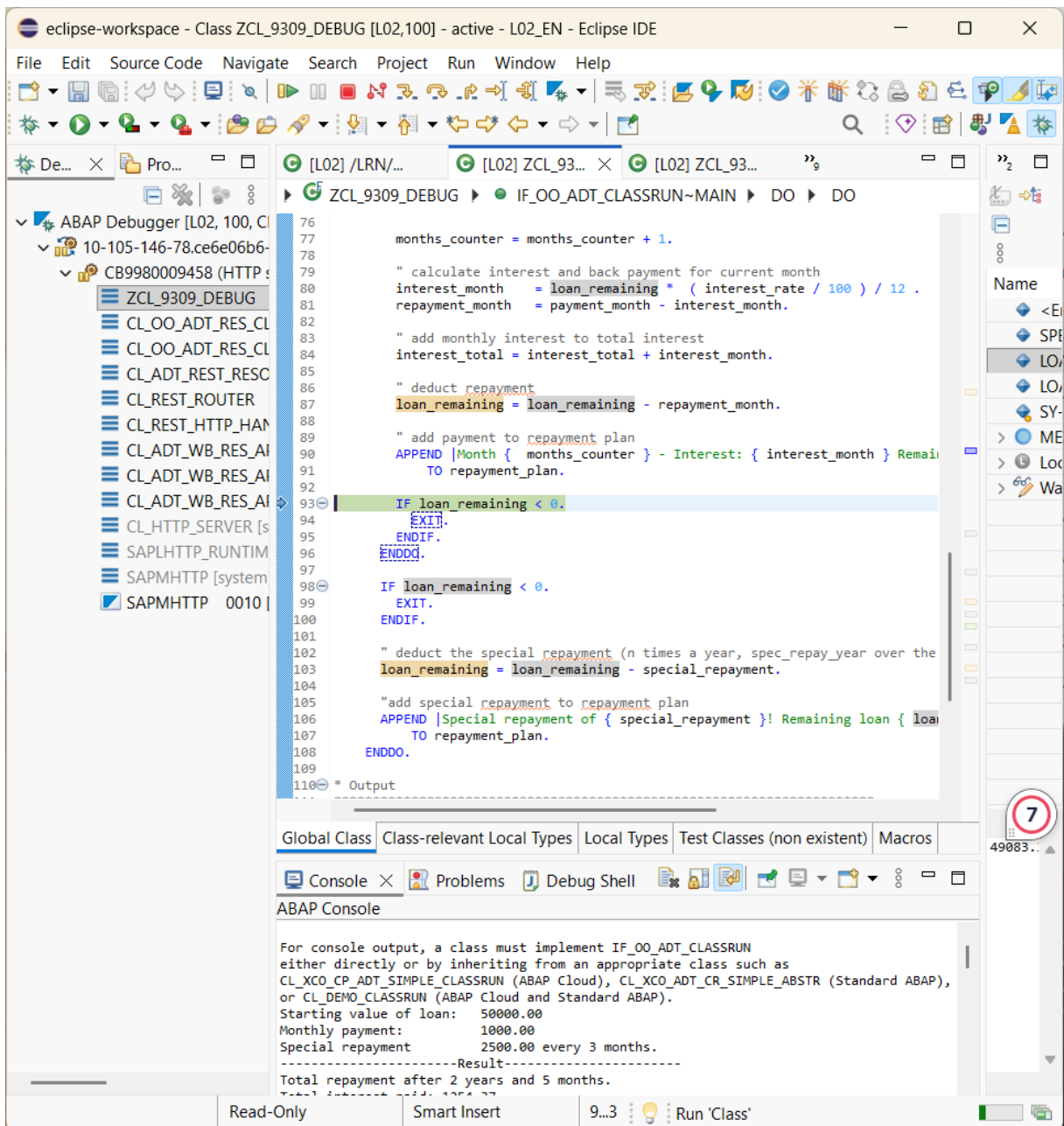
Screenshot: Debugger stopped at the loan repayment calculation.



Screenshot: F6 used during debugging; execution progressed through the repayment loop.



Screenshot: Watchpoint caused execution to stop when LOAN_REMAINING changed.



Screenshot: APPEND statement reached after the loan balance changed.

6. Task 2 – Analyze the Starting Values

The first executable statement is the assignment that initializes the remaining loan:

`loan_remaining = loan_total.`

A breakpoint was placed on this statement. When F9 was pressed, the debugger stopped before the assignment executed.

At that moment the important values were:

Variable	Value before assignment
LOAN_TOTAL	50000.00
LOAN_REMAINING	0.00

After pressing F5 once, the assignment executed and LOAN_REMAINING became 50000.00. This demonstrates how the debugger can show a variable changing immediately after a statement executes.

7. Task 3 – Control Program Execution

Step 1 – Execute the initialization:

F5 was used to execute `loan_remaining = loan_total`. The remaining loan changed from 0.00 to 50000.00.

Step 2 – Analyze the CASE:

SPEC_REPAY_MODE contains Q. The CASE statement therefore selects WHEN 'Q'. This sets MONTHS_BTW_SPEC_PAY to 3 and calculates SPECIAL_REPAYMENT as `SPEC_REPAY_YEAR / 4`.

Result:

`MONTHS_BTW_SPEC_PAY = 3`

`SPECIAL_REPAYMENT = 2500.00`

Step 3 – Use a watchpoint:

A watchpoint was created for LOAN_REMAINING. F8 resumed execution until the value changed. The debugger stopped around the statement:

`loan_remaining = loan_remaining - repayment_month.`

Step 4 – Inspect the repayment plan:

The APPEND statement adds a formatted text row to the internal table REPAYMENT_PLAN:

`APPEND |Month { months_counter } - Interest: { interest_month } Remaining loan: { loan_remaining } | TO repayment_plan.`

Step 5 – Observe loop values:

F6 was used to move through the calculation. The Locals view showed values such as `MONTHS_COUNTER = 3`, `LOAN_REMAINING = 48165.14`, `SPECIAL_REPAYMENT = 2500.00` and `MONTHS_BTW_SPEC_PAY = 3`. These values confirmed that the loop and calculations were executing.

8. Why the Debugger is Important in Real Companies

- It helps developers find the exact statement where a wrong value appears.
- It helps verify calculations without adding temporary output statements everywhere.
- It helps investigate loops, CASE branches, internal tables and conditions.
- It is useful for production-like troubleshooting when a program works but produces incorrect results.
- It reduces guesswork: instead of asking 'why is the result wrong?', we can inspect the values at each execution point.

9. Exercise 7 – Final Checklist

- Copied the debugging template ✓
- Created ZCL_9309_DEBUG ✓
- Activated and executed the class ✓
- Created and used a breakpoint ✓
- Entered the ABAP debugger ✓
- Inspected Variables/Locals ✓
- Used F5/F6 for controlled execution ✓
- Used a watchpoint for LOAN_REMAINING ✓
- Observed internal table REPAYMENT_PLAN ✓
- Verified changing loan calculation values ✓
- Exercise 7 completed ✓

10. Key Takeaway

The most important skill from Exercise 7 is not memorizing debugger buttons. It is learning to follow program execution and verify the data at the exact point where a decision or calculation happens.